



laC - Terraform

What Is Terraform State?

- As you were using Terraform to create and update resources, you might have noticed that every time you ran `terraform plan` or `terraform apply`, Terraform was able to find the resources it created previously and update them accordingly. But how did Terraform know which resources it was supposed to manage?
- Every time you run Terraform, it records information about what infrastructure it created in a Terraform state file. By default, when you run Terraform in the folder `/foo/bar`, Terraform creates the file `/foo/bar/terraform.tfstate`.
- This file contains a custom JSON format that records a mapping from the Terraform resources in your configuration files to the representation of those resources in the real world.

What Is Terraform State?

- A snippet of the contents of the terraform.tfstate file
- Using this JSON format, Terraform knows that a resource with type
- aws_instance and name example corresponds to an EC2 Instance in
- your AWS account with ID i-0bc4bbe5b84387543.
- Every time you run Terraform, it can fetch the latest status of this EC2 Instance from AWS and compare that to what's in your Terraform configurations to determine what changes need to be applied.
- In other words, the output of the plan command is a diff between the code on your computer and the infrastructure deployed in the real world, as discovered via IDs in the state file.

```
"version": 4,
"terraform_version": "1.2.3",
"serial": 1,
"lineage": "86545604-7463-4aa5-e9e8-a2a221de98d2",
"outputs": {},
"resources": [
  {
    "mode": "managed",
    "type": "aws_instance",
    "name": "example",
    "provider":
      "provider[\"registry.terraform.io/hashicorp/aws\"]",
    "instances": [
      {
        "schema_version": 1,
        "attributes": {
          "ami": "ami-0fb653ca2d3203ac1",
          "availability_zone": "us-east-2b",
          "id": "i-0bc4bbe5b84387543",
          "instance_state": "running",
          "instance_type": "t2.micro",
          "(...)": "(truncated)"
        }
      }
    ]
  }
]
```

Terraform State at scale

If you're using Terraform for a personal project, storing state in a single terraform.tfstate file that lives locally on your computer works just fine.

But if you want to use Terraform as a team on a real product, you run into several problems:

Shared storage for state files

To be able to use Terraform to update your infrastructure, each of your team members needs access to the same Terraform state files. That means you need to store those files in a shared location.

Locking state files

As soon as data is shared, you run into a new problem: locking. Without locking, if two team members are running Terraform at the same time, you can run into race conditions as multiple Terraform processes make concurrent updates to the state files, leading to conflicts, data loss, and state file corruption.

Isolating state files

When making changes to your infrastructure, it's a best practice to isolate different environments. For example, when making a change in a testing or staging environment, you want to be sure that there is no way you can accidentally break production.

Terraform State at scale

The most common technique for allowing multiple team members to access a common set of files is to put them in version control (e.g., Git). Although you should definitely store your Terraform code in version control, storing Terraform state in version control is a bad idea for the following reasons:

Manual error: It's too easy to forget to pull down the latest changes from version control before running Terraform or to push your latest changes to version control after running Terraform. It's just a matter of time before someone on your team runs Terraform with out-of-date state files and, as a result, accidentally rolls back or duplicates previous deployments.

Locking: Most version control systems do not provide any form of locking that would prevent two team members from running terraform apply on the same state file at the same time.

Secrets: All data in Terraform state files is stored in plain text. This is a problem because certain Terraform resources need to store sensitive data. For example, if you use the `aws_db_instance` resource to create a database, Terraform will store the username and password for the database in a state file in plain text, and you shouldn't store plain text secrets in version control.

Terraform State at scale

Instead of using version control, the best way to manage shared storage for state files is to use Terraform's built-in support for remote backends.

A Terraform backend determines how Terraform loads and stores state. The default backend, which you've been using this entire time, is the local backend, which stores the state file on your local disk.

Remote backends allow you to store the state file in a remote, shared store. A number of remote backends are supported, including Amazon S3, Azure Storage, Google Cloud Storage, and HashiCorp's Terraform Cloud and Terraform Enterprise. Remote backends solve the three issues just listed:

Manual error

After you configure a remote backend, Terraform will automatically load the state file from that backend every time you run `plan` or `apply`, and it'll automatically store the state file in that backend after each `apply`, so there's no chance of manual error.

Terraform State at scale

Locking

Most of the remote backends natively support locking. To run terraform apply, Terraform will automatically acquire a lock; if someone else is already running apply, they will already have the lock, and you will have to wait. You can run apply with the `-locktimeout=<TIME>` parameter to instruct Terraform to wait up to TIME for a lock to be released (e.g., `-lock-timeout=10m` will wait for 10 minutes).

Secrets

Most of the remote backends natively support encryption in transit and encryption at rest of the state file. Moreover, those backends usually expose ways to configure access permissions (e.g., using IAM policies with an Amazon S3 bucket), so you can control who has access to your state files and the secrets they might contain.

Terraform State at scale

To enable remote state storage with Amazon S3, the first step is to create an S3 bucket. Create a main.tf file in a new folder

Bucket: This is the name of the S3 bucket

prevent_destroy to true on a resource, any attempt to delete that resource (e.g., by running terraform destroy) will cause Terraform to exit with an error.

```
provider "aws" {  
    region = "us-east-2"  
}  
  
resource "aws_s3_bucket" "terraform_state" {  
    bucket = "terraform-up-and-running-state"  
  
    # Prevent accidental deletion of this S3 bucket  
    lifecycle {  
        prevent_destroy = true  
    }  
}
```


Terraform State at scale

Let's now add several extra layers of protection to this S3 bucket.

- First, use the `aws_s3_bucket_versioning` resource to enable versioning on the S3 bucket so that every update to a file in the bucket actually creates a new version of that file.
- This allows you to see older versions of the file and revert to those older versions at any time, which can be a useful fallback mechanism if something goes wrong

```
# Enable versioning so you can see the full revision history of  
your  
# state files  
resource "aws_s3_bucket_versioning" "enabled" {  
    bucket = aws_s3_bucket.terraform_state.id  
    versioning_configuration {  
        status = "Enabled"  
    }  
}
```

Terraform State at scale

- Second, use the `aws_s3_bucket_server_side_encryption_configuration` resource to turn server-side encryption on by default for all data written to this S3 bucket. This ensures that your state files, and any secrets they might contain, are always encrypted on disk when stored in S3:

```
# Enable server-side encryption by default
resource "aws_s3_bucket_server_side_encryption_configuration"
"default" {
    bucket = aws_s3_bucket.terraform_state.id

    rule {
        apply_server_side_encryption_by_default {
            sse_algorithm = "AES256"
        }
    }
}
```

Terraform State at scale

- Third, use the `aws_s3_bucket_public_access_block` resource to block all public access to the S3 bucket. S3 buckets are private by default, but as they are often used to serve static content—e.g., images, fonts, CSS, JS, HTML—it is possible, even easy, to make the buckets public.
- Since your Terraform state files may contain sensitive data and secrets, it's worth adding this extra layer of protection to ensure no one on your team can ever accidentally make this S3 bucket public:

```
# Explicitly block all public access to the S3 bucket
resource "aws_s3_bucket_public_access_block" "public_access" {
  bucket                = aws_s3_bucket.terraform_state.id
  block_public_acls      = true
  block_public_policy    = true
  ignore_public_acls     = true
  restrict_public_buckets = true
}
```

Terraform State at scale

- To configure Terraform to store the state in your S3 bucket (with encryption and locking), you need to add a backend configuration to your Terraform code. This is configuration for Terraform itself, so it resides within a terraform block and has the following syntax:
- where BACKEND_NAME is the name of the backend you want to use (e.g., "s3") and CONFIG consists of one or more arguments that are specific to that backend

```
terraform {  
  backend "<BACKEND_NAME>" {  
    [CONFIG...]  
  }  
}
```

Terraform State at scale

- Next, you need to create a DynamoDB table to use for locking. DynamoDB is Amazon's distributed key-value store. It supports strongly consistent reads and conditional writes, which are all the ingredients you need for a distributed lock system.
- Moreover, it's completely managed, so you don't have any infrastructure to run yourself, and it's inexpensive, with most Terraform usage easily fitting into the AWS Free Tier.
- To use DynamoDB for locking with Terraform, you must create a DynamoDB table that has a primary key called LockID (with this exact spelling and capitalization).

```
resource "aws_dynamodb_table" "terraform_locks" {
  name           = "terraform-up-and-running-locks"
  billing_mode   = "PAY_PER_REQUEST"
  hash_key       = "LockID"

  attribute {
    name = "LockID"

    type = "S"
  }
}
```

Terraform State at scale

- To configure Terraform to store the state in your S3 bucket (with encryption and locking), you need to add a backend configuration to your Terraform code. This is configuration for Terraform itself, so it resides within a terraform block and has the following syntax:
- where BACKEND_NAME is the name of the backend you want to use (e.g., "s3") and CONFIG consists of one or more arguments that are specific to that backend

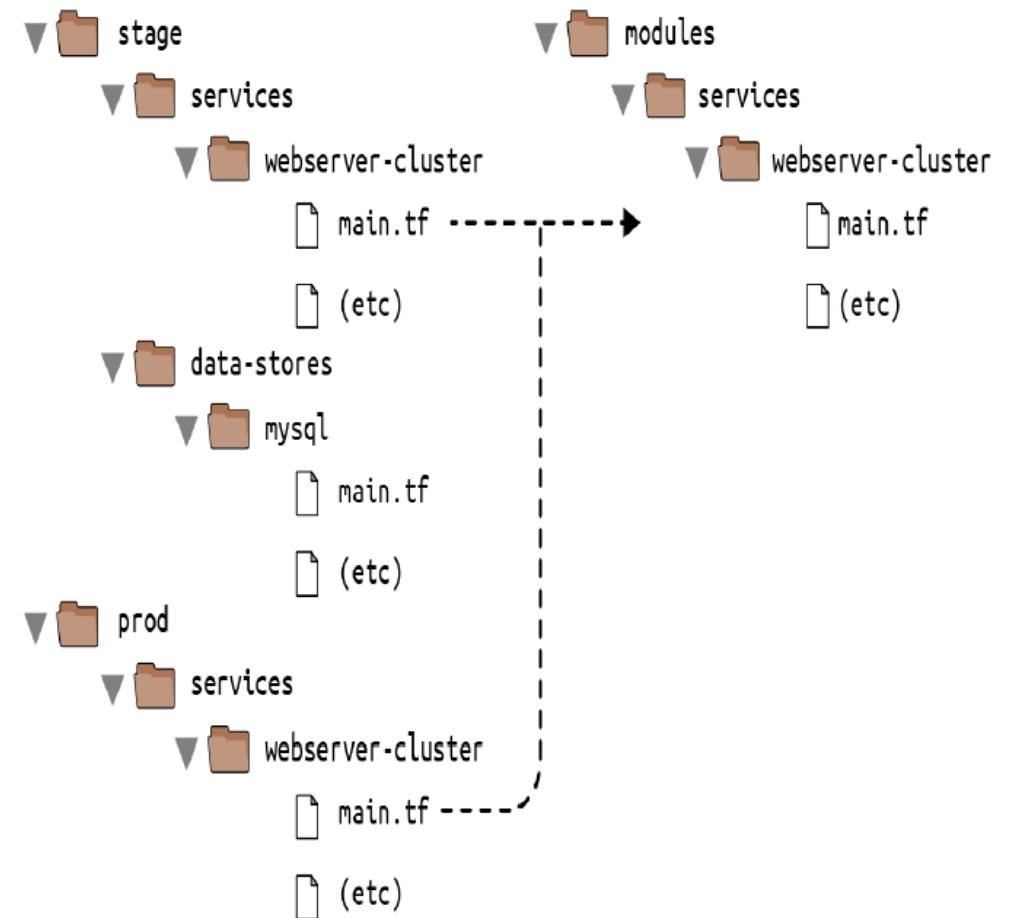
```
terraform {  
  backend "s3" {  
    # Replace this with your bucket name!  
    bucket      = "terraform-up-and-running-state"  
    key         = "global/s3/terraform.tfstate"  
    region      = "us-east-2"  
  
    # Replace this with your DynamoDB table name!  
    dynamodb_table = "terraform-up-and-running-locks"  
    encrypt        = true  
  }  
}
```

Terraform State at scale

- **Key:** The filepath within the S3 bucket where the Terraform state file should be written. You'll see a little later on why the preceding example code sets this to `global/s3/terraform.tfstate`.
- **Region:** The AWS region where the S3 bucket lives. Make sure to replace this with the region of the S3 bucket you created earlier.
- **dynamodb_table:** The DynamoDB table to use for locking. Make sure to replace this with the name of the DynamoDB table you created earlier.
- **Encrypt:** Setting this to true ensures that your Terraform state will be encrypted on disk when stored in S3. We already enabled default encryption in the S3 bucket itself, so this is here as a second layer to ensure that the data is always encrypted.
- To instruct Terraform to store your state file in this S3 bucket, you're going to use the `terraform init` command again. This command not only can download provider code, but also configure your Terraform backend.
- After running this command, your Terraform state will be stored in the S3 bucket. You can check this by heading over to the S3 Management Console in your browser and clicking your bucket.

How to Create Reusable Infrastructure with Terraform Modules

- In a general-purpose programming language such as Ruby, if you had the same code copied and pasted in several places, you could put that code inside of a function and reuse that function everywhere
- With Terraform, you can put your code inside of a Terraform module and reuse that module in multiple places throughout your code. Instead of having the same code copied and pasted in the staging and production environments, you'll be able to have both environments reuse code from the same module



Syntax of the Terraform Module Block

- Within your primary Terraform setup, you can call and instantiate modules using the modules block. The syntax is:

```
module "module_name" {  
  source = "path_of_module"  
  [other_arguments]  
}
```

- `module_name`: The user-defined name for the module instance is `module_name`.
- `source`: This identifies the module's location. A remote module registry (like GitHub or Terraform Registry) or a local file path could both be used.
- `other_arguments`: Optional arguments which module can accept as optional arguments.

Syntax of the Terraform Module Block

- Let's say we want to set up and provision EC2 instance using a module:
- In a different directory, we first define a module called "ec2_instance" with the following structure:
- Typical Module Structure:
- ec2_instance
 - └─ main.tf # Defines resources
 - └─ variables.tf # Declares input variables
 - └─ outputs.tf # Exposes resource outputs
- main.tf: Contains the core resource definitions required to provision infrastructure.
- variables.tf: Defines input variables such as instance type or AMI ID, allowing the module to be flexible and reusable.
- outputs.tf: Specifies the values the module should expose, such as resource IDs or public IP addresses.

Syntax of the Terraform Module Block

- we can instantiate this module as follows in our main Terraform configuration file:

```
provider "aws" {  
    region = "us-east-2"  
}  
  
module "my_server" {  
    source = "./ec2_instance"  
    instance_type = "t2.micro"  
    ami_id      = "ami-98656754364"  
}  
  
output "instance_id" {  
    value = module.gfg_server.instance_id  
}
```

Syntax of the Terraform Module Block

The content of the EC2 module `ec2_instance/main.tf`

```
resource "aws_instance" "this" {  
    ami      = var.ami_id  
    instance_type = var.instance_type  
    tags = {  
        Name = "Module-EC2-Instance" }  
}
```

ec2_instance/variables.tf

```
variable "instance_type" {  
    description = "Type of EC2 instance"  
    type        = string}  
  
variable "ami_id" {  
    description = "AMI ID for the EC2 instance"  
    type        = string}
```

ec2_instance/outputs.tf

```
output "instance_id" {  
    description = "The ID of the EC2 instance"  
    value       = aws_instance.this.id  
}
```